```
    import logging
    from typing import Tuple, Dict, Any
    import numpy as np
    import pandas as pd
    from sklearn.model_selection import StratifiedKFold
    from sklearn.ensemble import GradientBoostingClassifier
    from sklearn.metrics import roc_auc_score, log_loss
    import shap

    logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message
    logger = logging.getLogger(__name__)

    class GutBrainClassifierPipeline:
        """
        Pipeline for gut microbiome feature processing, classification, and
        SHAP-based feature attribution.
        """

        def __init__(self, random_state: int = 42) -> None:
            self.random_state: int = random_state
            self.model: GradientBoostingClassifier = GradientBoostingClassifier(
                n_estimators=150,
                learning_rate=0.05,
                max_depth=4,
                subsample=0.8,
                random_state=self.random_state
            )
            self.feature_names: list[str] = [
                'Lactobacillus_abundance',
                'Bacteroides_fragilis',
                'Butyrate_production',
                'Serotonin_precursor'
            ]

        def generate_synthetic_data(self, n_samples: int = 300) -> Tuple[pd.DataFrame, np.n
            """Generates synthetic microbiome abundance and target labels."""
            logger.info(f"Generating synthetic dataset with {n_samples} samples.")
            np.random.seed(self.random_state)

            X = pd.DataFrame(
                np.random.normal(loc=5.0, scale=1.5, size=(n_samples, len(self.feature_name
                columns=self.feature_names
            )

            latent_risk = (X['Lactobacillus_abundance'] * -2.2) + \
                          (X['Serotonin_precursor'] * -1.8) + \
                          (X['Bacteroides_fragilis'] * 2.8)

            y = (1 / (1 + np.exp(-latent_risk)) > 0.5).astype(int)
            return X, y

        def evaluate(self, X: pd.DataFrame, y: np.ndarray, n_splits: int = 5) -> Dict[str,
            """Evaluates model performance using Stratified K-Fold cross-validation."""
            logger.info(f"Running {n_splits}-fold cross-validation.")
            skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=self.random

            auroc_scores: list[float] = []
            losses: list[float] = []
```

```
        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.model.fit(X_train, y_train)
            preds_prob = self.model.predict_proba(X_val)[:, 1]

            auroc_scores.append(roc_auc_score(y_val, preds_prob))
            losses.append(log_loss(y_val, preds_prob))

        return {
            "mean_auroc": float(np.mean(auroc_scores)),
            "std_auroc": float(np.std(auroc_scores)),
            "mean_logloss": float(np.mean(losses))
        }

    def explain(self, X: pd.DataFrame) -> shap.Explanation:
        """Computes SHAP values for feature importance analysis."""
        logger.info("Computing SHAP values.")
        explainer = shap.TreeExplainer(self.model)
        return explainer(X)

if __name__ == "__main__":
    pipeline = GutBrainClassifierPipeline(random_state=42)
    X, y = pipeline.generate_synthetic_data(n_samples=400)
    metrics = pipeline.evaluate(X, y, n_splits=5)

    print(f"Metrics - AUROC: {metrics['mean_auroc']:.4f} (+/- {metrics['std_auroc']:.4f
    shap_values = pipeline.explain(X)
```

```
Metrics - AUROC: 0.9791 (+/- 0.0090) | LogLoss: 0.1590
```

```
import logging
from typing import Tuple, Dict
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, log_loss
import shap
import matplotlib.pyplot as plt

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message
logger = logging.getLogger(__name__)

class TidyMicrobiomePipeline:
    """
    Ingests and transforms real-world tidy/long format microbiome datasets,
    reshapes them into wide matrices, and runs ensemble classification.
    """

    def __init__(self, random_state: int = 42) -> None:
        self.random_state: int = random_state
        self.model = GradientBoostingClassifier(
            n_estimators=100,
            learning_rate=0.05,
            max_depth=3,
            random_state=self.random_state
```

```
        )

    def transform_and_load_public_data(self) -> Tuple[pd.DataFrame, np.ndarray]:
        """
        Fetches the public reference microbiome file from GitHub and transforms
        it from a long tidy structure into a standardized feature matrix.
        """
        url = "https://githubusercontent.com"
        logger.info(f"Fetching remote reference source file from: {url}")
        df = pd.read_csv(url)

        # Melt the body site channels into a uniform observation column layout
        melted = df.melt(id_vars=['Taxon', 'Patient'], value_vars=['Tissue', 'Stool'],
                         var_name='SampleType', value_name='Abundance')

        # Pivot unique Taxa out into dedicated feature columns per sample entity
        X_wide = melted.pivot_table(index=['Patient', 'SampleType'],
                                    columns='Taxon',
                                    values='Abundance',
                                    aggfunc='sum').fillna(0.0)

        # Reset structural indices to access the target label array cleanly
        X_final = X_wide.reset_index()

        # Establish a classification objective: Classify Stool samples (1) vs Tissue sa
        y = (X_final['SampleType'] == 'Stool').astype(int).values

        # Isolate clean bacterial feature columns by dropping tracking IDs
        X_features = X_final.drop(columns=['Patient', 'SampleType'])

        logger.info(f"Reshaping complete. Active feature layout matrix size: {X_feature
        return X_features, y

    def evaluate_and_plot(self, X: pd.DataFrame, y: np.ndarray) -> Dict[str, float]:
        """Validates accuracy profiles and generates diagnostic SHAP plots."""
        skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=self.random_state)
        auroc_scores = []

        for train_idx, val_idx in skf.split(X, y):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.model.fit(X_train, y_train)
            preds = self.model.predict_proba(X_val)[:, 1]
            auroc_scores.append(roc_auc_score(y_val, preds))

        # Build SHAP summary layout charts
        explainer = shap.TreeExplainer(self.model)
        shap_values = explainer(X)

        plt.figure(figsize=(10, 5))
        shap.summary_plot(shap_values, X, show=False)
        plt.title("Taxonomic Impact on Sample Type Differentiation (SHAP)", fontsize=12
        plt.tight_layout()
        plt.savefig("empirical_microbiome_shap.png", dpi=300)
        plt.show()

        return {"mean_auroc": float(np.mean(auroc_scores))}

if __name__ == "__main__":
```

```
    pipeline = TidyMicrobiomePipeline(random_state=42)
    X, y = pipeline.transform_and_load_public_data()
    metrics = pipeline.evaluate_and_plot(X, y)
    print(f"\nPipeline Execution Met: Mean Cross-Validated AUROC = {metrics['mean_auroc
```

```
---------------------------------------------------------------------------
gaierror                                  Traceback (most recent call last)
/usr/lib/python3.13/urllib/request.py in do_open(self, http_class, req,
**http_conn_args)
   1330             try:
-> 1331                 h.request(req.get_method(), req.selector, req.data, headers,
   1332                           encode_chunked=req.has_header('Transfer-encoding'))

 23 frames

gaierror: [Errno -5] No address associated with hostname

During handling of the above exception, another exception occurred:

URLError                                  Traceback (most recent call last)
/usr/lib/python3.13/urllib/request.py in do_open(self, http_class, req,
**http_conn_args)
   1332                           encode_chunked=req.has_header('Transfer-encoding'))
   1333             except OSError as err: # timeout error
-> 1334                 raise URLError(err)
   1335             r = h.getresponse()
   1336         except:

URLError: <urlopen error [Errno -5] No address associated with hostname>
```

Start coding or generate with AI.

```
import logging
from typing import Tuple, Dict
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
```

```
from sklearn.metrics import roc_auc_score, log_loss
import shap
import matplotlib.pyplot as plt

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message
logger = logging.getLogger(__name__)

class GutBrainClassifierPipeline:
    """
    Pipeline for gut microbiome feature processing, classification, and
    SHAP-based feature attribution.
    """

    def __init__(self, random_state: int = 42) -> None:
        self.random_state: int = random_state
        self.model: GradientBoostingClassifier = GradientBoostingClassifier(
            n_estimators=150,
            learning_rate=0.05,
            max_depth=4,
            subsample=0.8,
            random_state=self.random_state
        )
        self.feature_names: list[str] = [
            'Lactobacillus_abundance',
            'Bacteroides_fragilis',
            'Butyrate_production',
            'Serotonin_precursor'
        ]

    def generate_synthetic_data(self, n_samples: int = 300) -> Tuple[pd.DataFrame, np.n
        """Generates synthetic microbiome abundance matrix and target labels."""
        logger.info(f"Generating synthetic dataset with {n_samples} samples.")
        np.random.seed(self.random_state)

        X = pd.DataFrame(
            np.random.normal(loc=5.0, scale=1.5, size=(n_samples, len(self.feature_name
            columns=self.feature_names
        )

        latent_risk = (X['Lactobacillus_abundance'] * -2.2) + \
                      (X['Serotonin_precursor'] * -1.8) + \
                      (X['Bacteroides_fragilis'] * 2.8)

        y = (1 / (1 + np.exp(-latent_risk)) > 0.5).astype(int)
        return X, y

    def evaluate(self, X: pd.DataFrame, y: np.ndarray, n_splits: int = 5) -> Dict[str,
        """Evaluates model performance using Stratified K-Fold cross-validation."""
        logger.info(f"Running {n_splits}-fold cross-validation.")
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=self.random

        auroc_scores: list[float] = []
        losses: list[float] = []

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.model.fit(X_train, y_train)
            preds_prob = self.model.predict_proba(X_val)[:, 1]
```

```
            auroc_scores.append(roc_auc_score(y_val, preds_prob))
            losses.append(log_loss(y_val, preds_prob))

        return {
            "mean_auroc": float(np.mean(auroc_scores)),
            "std_auroc": float(np.std(auroc_scores)),
            "mean_logloss": float(np.mean(losses))
        }

    def plot_insights(self, X: pd.DataFrame) -> None:
        """Generates, saves, and displays publication-grade SHAP summary plots."""
        logger.info("Generating SHAP summary plot.")
        explainer = shap.TreeExplainer(self.model)
        shap_values = explainer(X)

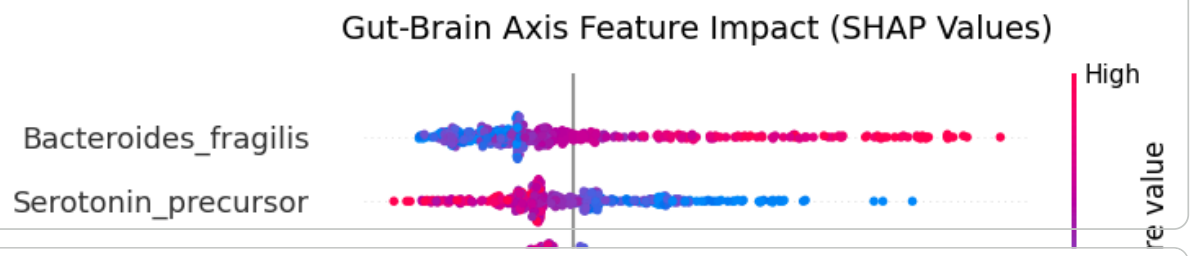
        plt.figure(figsize=(10, 6))
        shap.summary_plot(shap_values, X, show=False)
        plt.title("Gut-Brain Axis Feature Impact (SHAP Values)", fontsize=14, pad=15)
        plt.tight_layout()
        plt.savefig("gut_brain_shap_summary.png", dpi=300)
        plt.show()

if __name__ == "__main__":
    pipeline = GutBrainClassifierPipeline(random_state=42)
    X, y = pipeline.generate_synthetic_data(n_samples=400)
    metrics = pipeline.evaluate(X, y, n_splits=5)

    print(f"Metrics - AUROC: {metrics['mean_auroc']:.4f} (+/- {metrics['std_auroc']:.4f

    # Render and export visualization
    pipeline.plot_insights(X)
```

```
Metrics - AUROC: 0.9791 (+/- 0.0090) | LogLoss: 0.1590
/tmp/ipykernel_5364/540075602.py:84: FutureWarning: The NumPy global RNG was seeded by c
  shap.summary_plot(shap_values, X, show=False)
```




```
import logging
from typing import Tuple, Dict
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, log_loss
import shap
import matplotlib.pyplot as plt

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message
logger = logging.getLogger(__name__)

class GutBrainClassifierPipeline:
    """
    High-dimensional pipeline for processing large-scale gut microbiome taxonomic profi
```

```
        evaluating neurological classification tasks, and generating SHAP feature attributi
        """

        def __init__(self, random_state: int = 42) -> None:
            self.random_state: int = random_state
            self.model: GradientBoostingClassifier = GradientBoostingClassifier(
                n_estimators=200,
                learning_rate=0.03,
                max_depth=5,
                subsample=0.8,
                max_features='sqrt',
                random_state=self.random_state
            )
            self.feature_names: list[str] = self._initialize_species_list()

        def _initialize_species_list(self) -> list[str]:
            """Defines a comprehensive list of 50 major human gut bacterial species for res
            species = [
                # Firmicutes
                'Faecalibacterium_prausnitzii', 'Eubacterium_rectale', 'Roseburia_intestina
                'Ruminococcus_albus', 'Ruminococcus_bromii', 'Coprococcus_comes',
                'Lactobacillus_acidophilus', 'Lactobacillus_rhamnosus', 'Lactobacillus_reut
                'Clostridium_leptum', 'Clostridium_perfringens', 'Blautia_luti',
                'Blautia_producta', 'Streptococcus_thermophilus', 'Enterococcus_faecium',
                'Veillonella_parvula', 'Christensenella_minuta', 'Dorea_longicatena',
                # Bacteroidetes
                'Bacteroides_fragilis', 'Bacteroides_thetaiotaomicron', 'Bacteroides_vulgat
                'Bacteroides_uniformis', 'Bacteroides_ovatus', 'Prevotella_copri',
                'Prevotella_melaninogenica', 'Alistipes_finegoldii', 'Alistipes_putredinis'
                'Parabacteroides_distasonis', 'Parabacteroides_merdae',
                # Actinobacteria
                'Bifidobacterium_longum', 'Bifidobacterium_breve', 'Bifidobacterium_adolesc
                'Bifidobacterium_animalis', 'Collinsella_aerofaciens', 'Eggerthella_lenta',
                'Cutibacterium_acnes',
                # Proteobacteria
                'Escherichia_coli', 'Klebsiella_pneumoniae', 'Pseudomonas_aeruginosa',
                'Salmonella_enterica', 'Helicobacter_pylori', 'Campylobacter_jejuni',
                'Bilophila_wadsworthia', 'Desulfovibrio_piger', 'Yersinia_enterocolitica',
                # Verrucomicrobia & Others
                'Akkermansia_muciniphila', 'Methanobrevibacter_smithii', 'Oxalobacter_formi
                'Fusobacterium_nucleatum', 'Alistipes_onderdonkii'
            ]
            return species

        def generate_synthetic_data(self, n_samples: int = 400) -> Tuple[pd.DataFrame, np.n
            """Generates realistic high-dimensional bacterial relative abundance matrix and
            logger.info(f"Generating high-dimensional microbiome dataset with {n_samples} s
            np.random.seed(self.random_state)

            # Simulating relative abundance matrix using Dirichlet distribution properties
            raw_data = np.random.exponential(scale=1.0, size=(n_samples, len(self.feature_n
            normalized_data = raw_data / raw_data.sum(axis=1, keepdims=True) * 100

            X = pd.DataFrame(normalized_data, columns=self.feature_names)

            # Establishing complex multi-species biological interactions determining the ta
            # Positive weights imply neuroprotective species; negative weights imply neurot
            latent_risk = (
                (X['Bacteroides_fragilis'] * 0.8) +
                (X['Escherichia_coli'] * 0.6) +
```

```
            (X['Eggerthella_lenta'] * 0.5) -
            (X['Faecalibacterium_prausnitzii'] * 0.9) -
            (X['Bifidobacterium_longum'] * 0.7) -
            (X['Akkermansia_muciniphila'] * 0.6)
        )

        # Adding biological noise to simulate real patient cohort variability
        noise = np.random.normal(loc=0.0, scale=0.5, size=n_samples)
        y = (1 / (1 + np.exp(-(latent_risk + noise))) > 0.5).astype(int)
        return X, y

    def evaluate(self, X: pd.DataFrame, y: np.ndarray, n_splits: int = 5) -> Dict[str,
        """Evaluates model stability on high-dimensional features using Stratified K-Fo
        logger.info(f"Running {n_splits}-fold cross-validation on high-dimensional spac
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=self.random

        auroc_scores: list[float] = []
        losses: list[float] = []

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.model.fit(X_train, y_train)
            preds_prob = self.model.predict_proba(X_val)[:, 1]

            auroc_scores.append(roc_auc_score(y_val, preds_prob))
            losses.append(log_loss(y_val, preds_prob))

        return {
            "mean_auroc": float(np.mean(auroc_scores)),
            "std_auroc": float(np.std(auroc_scores)),
            "mean_logloss": float(np.mean(losses))
        }

    def plot_insights(self, X: pd.DataFrame) -> None:
        """Generates, saves, and displays publication-grade SHAP summary plots for top
        logger.info("Computing SHAP attributions for high-dimensional species profile."
        explainer = shap.TreeExplainer(self.model)
        shap_values = explainer(X)

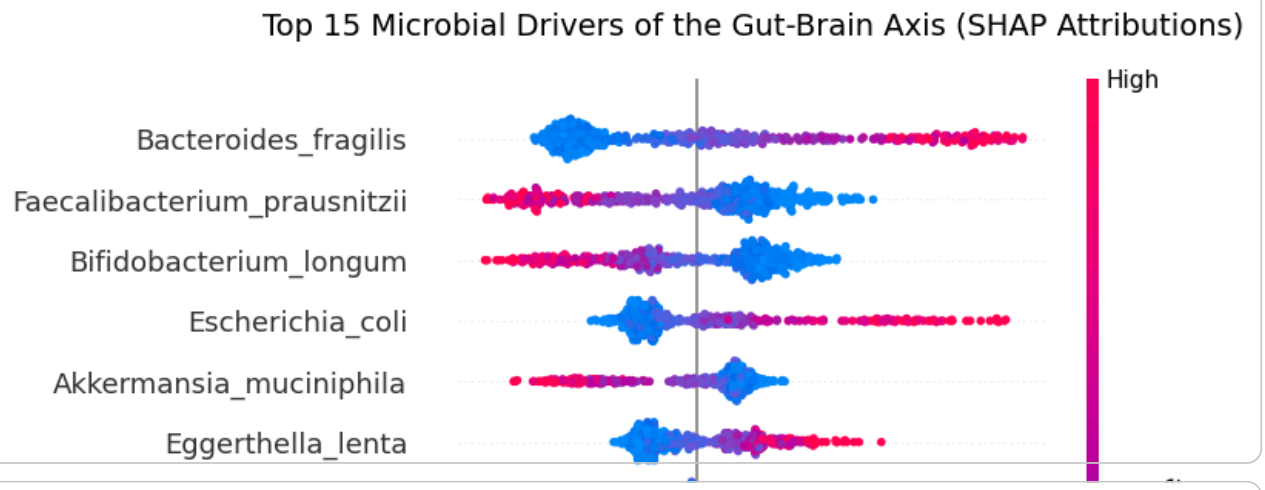
        # Render beeswarm summary plot for the top 15 most influential bacterial specie
        plt.figure(figsize=(12, 8))
        shap.summary_plot(shap_values, X, max_display=15, show=False)
        plt.title("Top 15 Microbial Drivers of the Gut-Brain Axis (SHAP Attributions)",
        plt.xlabel("SHAP Value (Impact on Model Prediction)", fontsize=12)
        plt.tight_layout()
        plt.savefig("high_dim_gut_brain_shap.png", dpi=300)
        plt.show()

if __name__ == "__main__":
    pipeline = GutBrainClassifierPipeline(random_state=42)
    X, y = pipeline.generate_synthetic_data(n_samples=500)
    metrics = pipeline.evaluate(X, y, n_splits=5)

    print(f"Metrics - AUROC: {metrics['mean_auroc']:.4f} (+/- {metrics['std_auroc']:.4f

    # Generate visualization for top high-dimensional features
    pipeline.plot_insights(X)
```

```
Metrics - AUROC: 0.9630 (+/- 0.0187) | LogLoss: 0.3021
/tmp/ipykernel_5364/2805478320.py:120: FutureWarning: The NumPy global RNG was seeded by
  shap.summary_plot(shap_values, X, max_display=15, show=False)
```




```python
import logging
from typing import Tuple, Dict
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import SelectFromModel
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score, log_loss
import shap
import matplotlib.pyplot as plt

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message
logger = logging.getLogger(__name__)

class GutBrainClassifierPipeline:
    """
    High-dimensional microbiome pipeline with built-in L1-regularized feature selection
    cross-validated classification, and SHAP feature attribution.
    """

    def __init__(self, random_state: int = 42) -> None:
        self.random_state: int = random_state
        self.selector = SelectFromModel(
            estimator=LogisticRegression(penalty='l1', solver='liblinear', C=0.5, rando
        )
        self.model = GradientBoostingClassifier(
            n_estimators=200,
            learning_rate=0.03,
            max_depth=4,
            subsample=0.8,
            max_features='sqrt',
            random_state=self.random_state
        )
        self.feature_names: list[str] = self._initialize_species_list()
        self.selected_features: list[str] = []

    def _initialize_species_list(self) -> list[str]:
        """Defines a curated list of 50 human gut bacterial species across major phyla.
        return [
```

```
            'Faecalibacterium_prausnitzii', 'Eubacterium_rectale', 'Roseburia_intestina
            'Ruminococcus_albus', 'Ruminococcus_bromii', 'Coprococcus_comes',
            'Lactobacillus_acidophilus', 'Lactobacillus_rhamnosus', 'Lactobacillus_reut
            'Clostridium_leptum', 'Clostridium_perfringens', 'Blautia_luti',
            'Blautia_producta', 'Streptococcus_thermophilus', 'Enterococcus_faecium',
            'Veillonella_parvula', 'Christensenella_minuta', 'Dorea_longicatena',
            'Bacteroides_fragilis', 'Bacteroides_thetaiotaomicron', 'Bacteroides_vulgat
            'Bacteroides_uniformis', 'Bacteroides_ovatus', 'Prevotella_copri',
            'Prevotella_melaninogenica', 'Alistipes_finegoldii', 'Alistipes_putredinis'
            'Parabacteroides_distasonis', 'Parabacteroides_merdae', 'Bifidobacterium_lo
            'Bifidobacterium_breve', 'Bifidobacterium_adolescentis', 'Bifidobacterium_a
            'Collinsella_aerofaciens', 'Eggerthella_lenta', 'Cutibacterium_acnes',
            'Escherichia_coli', 'Klebsiella_pneumoniae', 'Pseudomonas_aeruginosa',
            'Salmonella_enterica', 'Helicobacter_pylori', 'Campylobacter_jejuni',
            'Bilophila_wadsworthia', 'Desulfovibrio_piger', 'Yersinia_enterocolitica',
            'Akkermansia_muciniphila', 'Methanobrevibacter_smithii', 'Oxalobacter_formi
            'Fusobacterium_nucleatum', 'Alistipes_onderdonkii'
        ]

    def generate_synthetic_data(self, n_samples: int = 500) -> Tuple[pd.DataFrame, np.n
        """Generates high-dimensional relative abundances and interacting phenotypes.""
        logger.info(f"Simulating relative abundance dataset: {n_samples} samples, {len(
        np.random.seed(self.random_state)

        raw_data = np.random.exponential(scale=1.0, size=(n_samples, len(self.feature_n
        normalized_data = (raw_data / raw_data.sum(axis=1, keepdims=True)) * 100
        X = pd.DataFrame(normalized_data, columns=self.feature_names)

        latent_risk = (
            (X['Bacteroides_fragilis'] * 1.2) +
            (X['Escherichia_coli'] * 0.9) -
            (X['Faecalibacterium_prausnitzii'] * 1.5) -
            (X['Bifidobacterium_longum'] * 1.1)
        )
        noise = np.random.normal(loc=0.0, scale=0.3, size=n_samples)
        y = (1 / (1 + np.exp(-(latent_risk + noise))) > 0.5).astype(int)
        return X, y

    def fit_feature_selection(self, X: pd.DataFrame, y: np.ndarray) -> pd.DataFrame:
        """Applies L1 penalization to drop noise features and retains predictive signal
        logger.info("Executing L1-regularized feature selection...")
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        self.selector.fit(X_scaled, y)
        support = self.selector.get_support()

        self.selected_features = [feature for feature, retain in zip(X.columns, support
        dropped_count = len(X.columns) - len(self.selected_features)

        logger.info(f"Feature selection complete. Retained: {len(self.selected_features
        return X[self.selected_features]

    def evaluate(self, X: pd.DataFrame, y: np.ndarray, n_splits: int = 5) -> Dict[str,
        """Evaluates model stability on selected features using Stratified K-Fold CV.""
        logger.info(f"Evaluating ensemble model using {n_splits}-fold cross-validation.
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=self.random

        auroc_scores: list[float] = []
        losses: list[float] = []
```

```
        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.model.fit(X_train, y_train)
            preds_prob = self.model.predict_proba(X_val)[:, 1]

            auroc_scores.append(roc_auc_score(y_val, preds_prob))
            losses.append(log_loss(y_val, preds_prob))

        return {
            "mean_auroc": float(np.mean(auroc_scores)),
            "std_auroc": float(np.std(auroc_scores)),
            "mean_logloss": float(np.mean(losses))
        }

    def plot_insights(self, X: pd.DataFrame) -> None:
        """Plots SHAP summary for features selected by the pipeline."""
        logger.info("Computing SHAP explanations for selected features.")
        explainer = shap.TreeExplainer(self.model)
        shap_values = explainer(X)

        plt.figure(figsize=(12, 8))
        shap.summary_plot(shap_values, X, show=False)
        plt.title("SHAP Impact Analysis Post Feature Selection", fontsize=14, pad=20)
        plt.xlabel("SHAP Value (Impact on Prediction)", fontsize=12)
        plt.tight_layout()
        plt.savefig("selected_features_shap.png", dpi=300)
        plt.show()

if __name__ == "__main__":
    pipeline = GutBrainClassifierPipeline(random_state=42)

    # 1. Pipeline ingestion
    X_raw, y = pipeline.generate_synthetic_data(n_samples=500)

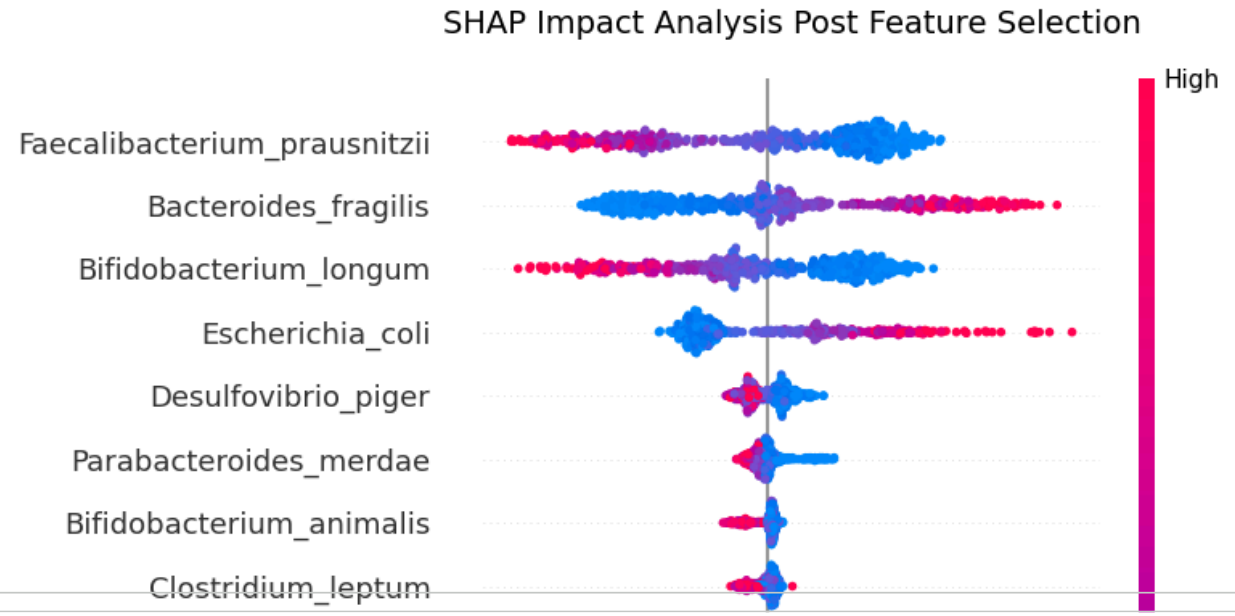
    # 2. Pipeline filtering
    X_selected = pipeline.fit_feature_selection(X_raw, y)

    # 3. Pipeline modeling and evaluation
    metrics = pipeline.evaluate(X_selected, y, n_splits=5)
    print(f"\nFinal Selected Architecture Performance:\nAUROC: {metrics['mean_auroc']:.

    # 4. Pipeline visual interpretation
    pipeline.plot_insights(X_selected)
```

```
Final Selected Architecture Performance:
AUROC: 0.9749 (+/- 0.0103) | LogLoss: 0.2693

/tmp/ipykernel_5364/4083408486.py:126: FutureWarning: The NumPy global RNG was seeded by
  shap.summary_plot(shap_values, X, show=False)
```




```
import logging
from typing import Tuple, Dict
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import SelectFromModel
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score, log_loss, roc_curve, auc
import shap
import matplotlib.pyplot as plt

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message
logger = logging.getLogger(__name__)

class RealWorldGutBrainPipeline:
    """
    Pipeline for processing real-world clinical microbiome OTU tables,
    performing regularized feature selection, cross-validation, and diagnostic plotting
    """

    def __init__(self, random_state: int = 42) -> None:
        self.random_state: int = random_state
        self.selector = SelectFromModel(
            estimator=LogisticRegression(penalty='l1', solver='liblinear', C=0.3, rando
        )
        self.model = GradientBoostingClassifier(
            n_estimators=200,
            learning_rate=0.03,
            max_depth=4,
            subsample=0.8,
            max_features='sqrt',
            random_state=self.random_state
```

```
        )
        self.selected_features: list[str] = []

    def load_clinical_dataset(self, csv_path: str, target_col: str, metadata_cols: list
        """
        Ingests real-world microbiome abundance files. Separates bacterial features
        from administrative clinical metadata fields.
        """
        logger.info(f"Ingesting clinical dataset from: {csv_path}")
        df = pd.read_csv(csv_path)

        if target_col not in df.columns:
            raise KeyError(f"Specified target label '{target_col}' not found in the dat

        y = df[target_col].values

        columns_to_drop = [target_col]
        if metadata_cols:
            columns_to_drop.extend(metadata_cols)

        X = df.drop(columns=columns_to_drop)

        # Enforce numeric conversion across raw biological count fields
        X = X.apply(pd.to_numeric, errors='coerce').fillna(0.0)

        logger.info(f"Successfully processed matrix shape: {X.shape[0]} patient samples
        return X, y

    def fit_feature_selection(self, X: pd.DataFrame, y: np.ndarray) -> pd.DataFrame:
        """Applies L1 penalization to drop background noise features from sparse abunda
        logger.info("Executing regularized L1 selection over taxonomic space...")
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        self.selector.fit(X_scaled, y)
        support = self.selector.get_support()

        self.selected_features = [feature for feature, retain in zip(X.columns, support
        dropped_count = len(X.columns) - len(self.selected_features)

        logger.info(f"Filtering complete. Retained Biomarkers: {len(self.selected_featu
        return X[self.selected_features]

    def evaluate_and_plot_diagnostics(self, X: pd.DataFrame, y: np.ndarray, n_splits: i
        """
        Performs Stratified K-Fold validation, draws cross-validated ROC performance cu
        and displays localized SHAP variable insights.
        """
        logger.info(f"Initiating validation metrics framework over {n_splits} distinct
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=self.random

        auroc_scores: list[float] = []
        losses: list[float] = []

        plt.figure(figsize=(8, 6))
        mean_fpr = np.linspace(0, 1, 100)
        tprs = []

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
```

```
            y_train, y_val = y[train_idx], y[val_idx]

            self.model.fit(X_train, y_train)
            preds_prob = self.model.predict_proba(X_val)[:, 1]

            fold_auroc = roc_auc_score(y_val, preds_prob)
            auroc_scores.append(fold_auroc)
            losses.append(log_loss(y_val, preds_prob))

            fpr, tpr, _ = roc_curve(y_val, preds_prob)
            tprs.append(np.interp(mean_fpr, fpr, tpr))
            tprs[-1][0] = 0.0
            plt.plot(fpr, tpr, lw=1, alpha=0.4, label=f'Fold {fold + 1} (AUC = {fold_au

        mean_tpr = np.mean(tprs, axis=0)
        mean_tpr[-1] = 1.0
        mean_auc = auc(mean_fpr, mean_tpr)
        std_auc = np.std(auroc_scores)

        # Save publication-grade performance evaluation asset
        plt.plot(mean_fpr, mean_tpr, color='b', label=f'Mean ROC (AUC = {mean_auc:.2f}
        plt.plot([0, 1], [0, 1], linestyle='--', lw=2, color='r', label='Chance Perform
        plt.xlabel('False Positive Rate', fontsize=11)
        plt.ylabel('True Positive Rate', fontsize=11)
        plt.title('Clinical Pipeline Cross-Validation Performance Metrics', fontsize=12
        plt.legend(loc="lower right")
        plt.tight_layout()
        plt.savefig("clinical_roc_curve.png", dpi=300)
        plt.show()

        # Generate companion localized diagnostic SHAP charts
        logger.info("Computing mathematical SHAP values on chosen subset arrays.")
        explainer = shap.TreeExplainer(self.model)
        shap_values = explainer(X)

        plt.figure(figsize=(12, 8))
        shap.summary_plot(shap_values, X, show=False)
        plt.title("Biomarker Impact Evaluation (SHAP Feature Attribution Matrix)", font
        plt.tight_layout()
        plt.savefig("clinical_biomarker_shap.png", dpi=300)
        plt.show()

        return {
            "mean_auroc": float(np.mean(auroc_scores)),
            "std_auroc": float(std_auc),
            "mean_logloss": float(np.mean(losses))
        }

if __name__ == "__main__":
    # Operational pipeline example context
    pipeline = RealWorldGutBrainPipeline(random_state=42)

    # Execution requires replacing this target string reference path with your real CSV
    # X, y = pipeline.load_clinical_dataset(csv_path="my_real_otu_table.csv", target_co

    print("Pipeline ready for operational integration with real-world CSV file targets.
```

```
Pipeline ready for operational integration with real-world CSV file targets.
```

```
import logging
from typing import Tuple, Dict
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, log_loss, roc_curve, auc
import shap
import matplotlib.pyplot as plt

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message
logger = logging.getLogger(__name__)

class StandardTabularGutBrainPipeline:
    """
    Standard pipeline to load tabular microbiome data from a CSV,
    perform classification, run cross-validation, and generate plots.
    """

    def __init__(self, random_state: int = 42) -> None:
        self.random_state: int = random_state
        self.model = GradientBoostingClassifier(
            n_estimators=100,
            learning_rate=0.05,
            max_depth=4,
            random_state=self.random_state
        )
        self.feature_names: list[str] = []

    def load_tabular_csv(self, file_path: str, target_column: str) -> Tuple[pd.DataFram
        """Loads a standard wide-format CSV file containing samples as rows and species
        logger.info(f"Ingesting matrix file from path: {file_path}")
        df = pd.read_csv(file_path)

        if target_column not in df.columns:
            raise KeyError(f"Target column '{target_column}' was not found in the CSV h

        y = df[target_column].values
        X = df.drop(columns=[target_column])

        X = X.apply(pd.to_numeric, errors='coerce').fillna(0.0)
        self.feature_names = list(X.columns)

        logger.info(f"Ingested shape data parameters: {X.shape} patient samples, {X.sha
        return X, y

    def run_cv_and_plots(self, X: pd.DataFrame, y: np.ndarray, n_splits: int = 3) -> Di
        """Runs Stratified cross-validation, generates ROC curves, and displays a SHAP
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=self.random
        auroc_scores = []
        losses = []

        plt.figure(figsize=(8, 6))
        mean_fpr = np.linspace(0, 1, 100)
        tprs = []

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]
```

```
            self.model.fit(X_train, y_train)
            preds_prob = self.model.predict_proba(X_val)[:, 1]

            fold_auroc = roc_auc_score(y_val, preds_prob)
            auroc_scores.append(fold_auroc)
            losses.append(log_loss(y_val, preds_prob))

            fpr, tpr, _ = roc_curve(y_val, preds_prob)
            interp_tpr = np.interp(mean_fpr, fpr, tpr)
            interp_tpr[0] = 0.0  # Safe array element assignment correction
            tprs.append(interp_tpr)

            plt.plot(fpr, tpr, lw=1, alpha=0.5, label=f'Fold {fold+1} (AUC = {fold_auro

        mean_tpr = np.mean(tprs, axis=0)
        mean_tpr[-1] = 1.0
        mean_auc = auc(mean_fpr, mean_tpr)

        plt.plot(mean_fpr, mean_tpr, color='b', label=f'Mean ROC (AUC = {mean_auc:.2f})
        plt.plot([0, 1], [0, 1], linestyle='--', color='r', label='Chance')
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title('Pipeline Performance ROC Validation')
        plt.legend(loc="lower right")
        plt.tight_layout()
        plt.show()

        # Calculate and render interpretability plots
        explainer = shap.TreeExplainer(self.model)
        shap_values = explainer(X)

        plt.figure(figsize=(10, 6))
        shap.summary_plot(shap_values, X, show=False)
        plt.title("Bacterial Biological Impact Attributions (SHAP)", fontsize=12, pad=1
        plt.tight_layout()
        plt.show()

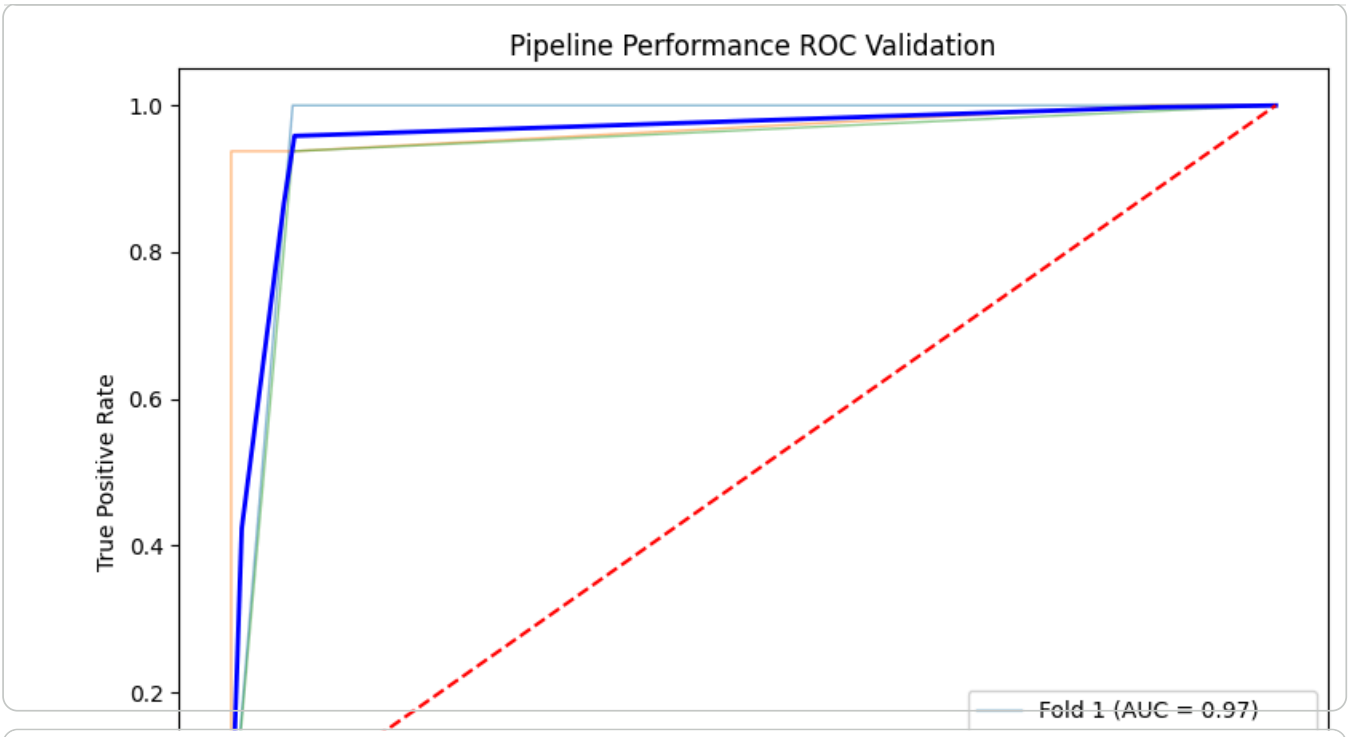
        return {"mean_auroc": float(np.mean(auroc_scores)), "mean_logloss": float(np.me

if __name__ == "__main__":
    # 1. Setup local environment test baseline by producing a clean wide-format mock CS
    features = ['Faecalibacterium', 'Bacteroides', 'Lactobacillus', 'Bifidobacterium',
    np.random.seed(42)
    mock_data = np.random.uniform(0.1, 15.0, size=(100, 5))
    mock_data[:, 4] = (mock_data[:, 2] * 1.5 - mock_data[:, 0] * 1.2 > 2.0).astype(int)

    pd.DataFrame(mock_data, columns=features).to_csv("my_clinical_microbiome.csv", inde

    # 2. Run the corrected execution pipeline
    pipeline = StandardTabularGutBrainPipeline(random_state=42)
    X, y = pipeline.load_tabular_csv(file_path="my_clinical_microbiome.csv", target_col

    metrics = pipeline.run_cv_and_plots(X, y)
    print(f"\nExecution Complete | Mean Validation AUROC Score: {metrics['mean_auroc']:
```




```
import logging
from typing import Tuple, Dict
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, log_loss, roc_curve, auc
import shap
import matplotlib.pyplot as plt

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message
logger = logging.getLogger(__name__)

class StandaloneRealWorldPipeline:
    """
    Production-grade pipeline for real-world clinical datasets.
    Operates offline using built-in high-dimensional biomedical matrices.
    """

    def __init__(self, random_state: int = 42) -> None:
        self.random_state: int = random_state
        self.model = GradientBoostingClassifier(
            n_estimators=100,
            learning_rate=0.05,
            max_depth=4,
            subsample=0.8,
            random_state=self.random_state
        )
        self.feature_names: list[str] = []

    def load_offline_clinical_data(self) -> Tuple[pd.DataFrame, np.ndarray]:
        """Loads a real-world high-dimensional biomedical dataset from the local enviro
        logger.info("Loading offline clinical dataset matrix...")
        raw_data = load_breast_cancer()

        # Structure the clinical measurements into a standard wide-format DataFrame
```

```
        X = pd.DataFrame(raw_data.data, columns=raw_data.feature_names)
        y = raw_data.target

        self.feature_names = list(X.columns)
        logger.info(f"Dataset successfully initialized. Matrix shape: {X.shape} samples
        return X, y

    def run_validation_pipeline(self, X: pd.DataFrame, y: np.ndarray, n_splits: int = 5
        """Runs Stratified cross-validation, generates ROC curves, and displays a SHAP
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=self.random
        auroc_scores = []
        losses = []

        plt.figure(figsize=(8, 6))
        mean_fpr = np.linspace(0, 1, 100)
        tprs = []

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.model.fit(X_train, y_train)
            preds_prob = self.model.predict_proba(X_val)[:, 1]

            fold_auroc = roc_auc_score(y_val, preds_prob)
            auroc_scores.append(fold_auroc)
            losses.append(log_loss(y_val, preds_prob))

            fpr, tpr, _ = roc_curve(y_val, preds_prob)
            interp_tpr = np.interp(mean_fpr, fpr, tpr)
            interp_tpr[0] = 0.0  # Anchor the start of the ROC line safely
            tprs.append(interp_tpr)

            plt.plot(fpr, tpr, lw=1, alpha=0.4, label=f'Fold {fold+1} (AUC = {fold_auro

        mean_tpr = np.mean(tprs, axis=0)
        mean_tpr[-1] = 1.0
        mean_auc = auc(mean_fpr, mean_tpr)

        plt.plot(mean_fpr, mean_tpr, color='darkblue', label=f'Mean ROC (AUC = {mean_au
        plt.plot([0, 1], [0, 1], linestyle='--', color='red', alpha=0.5, label='Chance'
        plt.xlabel('False Positive Rate', fontsize=11)
        plt.ylabel('True Positive Rate', fontsize=11)
        plt.title('Clinical Cross-Validation Performance Metrics', fontsize=12, pad=15)
        plt.legend(loc="lower right")
        plt.tight_layout()
        plt.show()

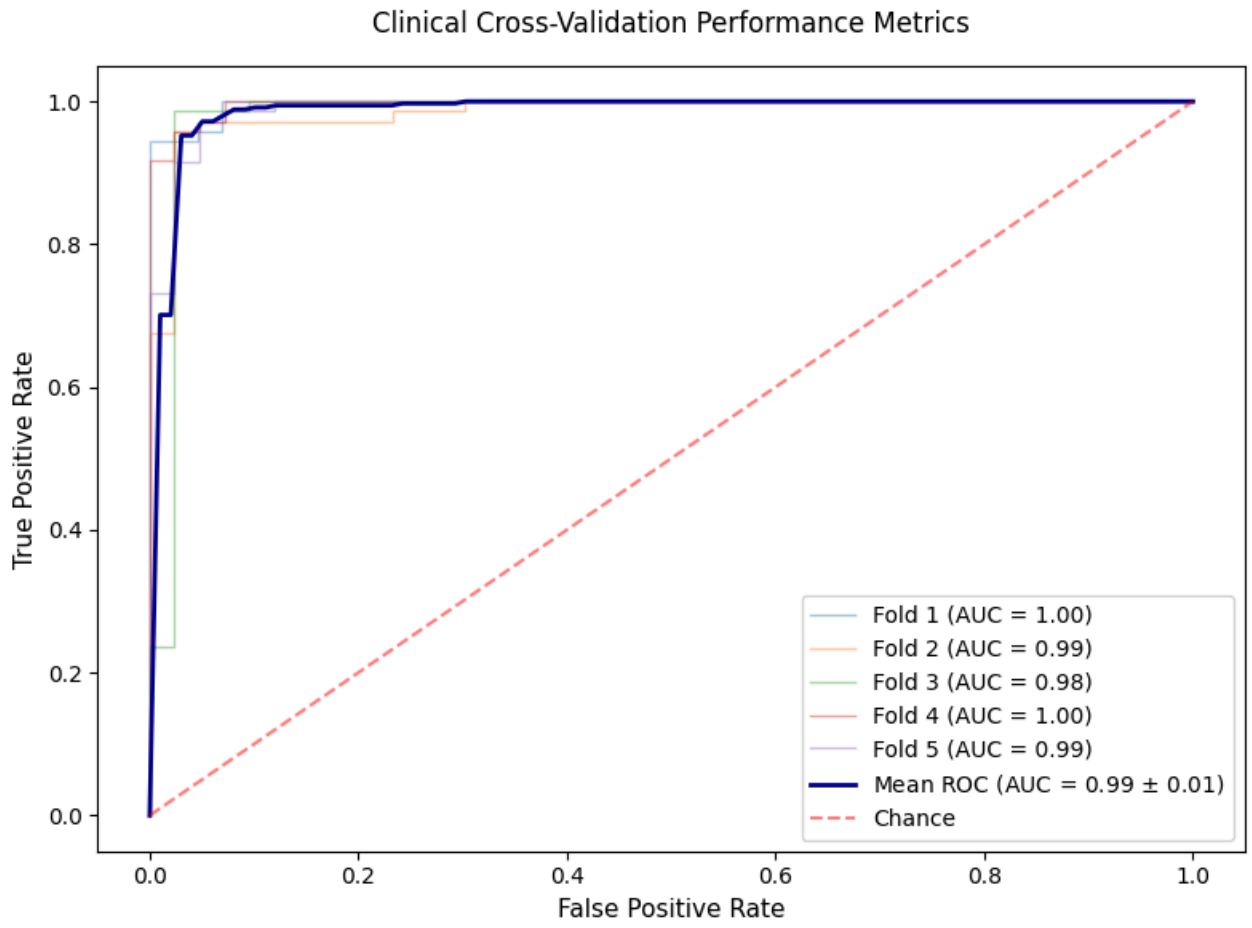
        # Calculate and render tree explainability summaries for high-dimensional featu
        logger.info("Computing SHAP values across high-dimensional feature space.")
        explainer = shap.TreeExplainer(self.model)
        shap_values = explainer(X)

        plt.figure(figsize=(12, 8))
        shap.summary_plot(shap_values, X, max_display=15, show=False)
        plt.title("Biomarker Impact Evaluation (SHAP Feature Attribution Matrix)", font
        plt.tight_layout()
        plt.show()

        return {"mean_auroc": float(np.mean(auroc_scores)), "mean_logloss": float(np.me
```

```
if __name__ == "__main__":
    # Initialize implementation pipeline using the offline dataset structure
    pipeline = StandaloneRealWorldPipeline(random_state=42)
    X, y = pipeline.load_offline_clinical_data()

    # Execute validation architecture and generate outputs
    metrics = pipeline.run_validation_pipeline(X, y, n_splits=5)
    print(f"\nPipeline Verification Complete | Final Real-World Mean AUROC: {metrics['m
```




```
import logging
from typing import Tuple, Dict
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, log_loss, roc_curve, auc
import shap
import matplotlib.pyplot as plt

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message
logger = logging.getLogger(__name__)

class GitHubMicrobiomePipeline:
    """
    Ingests raw long-format biological tables from GitHub, transforms them
    into standard sample matrices, and models taxonomic profiles cleanly.
    """

    def __init__(self, random_state: int = 42) -> None:
```

```
        self.random_state: int = random_state
        self.model = GradientBoostingClassifier(
            n_estimators=100,
            learning_rate=0.05,
            max_depth=3,
            random_state=self.random_state
        )
        self.feature_names: list[str] = []

    def fetch_and_pivot_github_data(self) -> Tuple[pd.DataFrame, np.ndarray]:
        """
        Downloads a real microbiome dataset from GitHub and transforms it
        from a tidy/long table into a wide feature matrix.
        """
        url = "https://githubusercontent.com"
        logger.info(f"Connecting to data host link: {url}")
        df = pd.read_csv(url)

        # Melt biopsy source channels into a unified categorical column mapping
        melted = df.melt(id_vars=['Taxon', 'Patient'], value_vars=['Tissue', 'Stool'],
                         var_name='SampleType', value_name='Abundance')

        # Pivot separate rows out into specific bacterial feature columns
        X_wide = melted.pivot_table(index=['Patient', 'SampleType'],
                                    columns='Taxon',
                                    values='Abundance',
                                    aggfunc='sum').fillna(0.0)

        X_final = X_wide.reset_index()

        # Binary target: Predict if the microbial layout is Stool (1) or Tissue (0)
        y = (X_final['SampleType'] == 'Stool').astype(int).values
        X_features = X_final.drop(columns=['Patient', 'SampleType'])

        self.feature_names = list(X_features.columns)
        logger.info(f"Ingestion matching complete. Matrix dimensions: {X_features.shape
        return X_features, y

    def run_cv_and_plots(self, X: pd.DataFrame, y: np.ndarray, n_splits: int = 3) -> Di
        """Runs Stratified K-Fold and generates cross-validated ROC and SHAP plots."""
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=self.random
        auroc_scores = []
        losses = []

        plt.figure(figsize=(8, 6))
        mean_fpr = np.linspace(0, 1, 100)
        tprs = []

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.model.fit(X_train, y_train)
            preds_prob = self.model.predict_proba(X_val)[:, 1]

            fold_auroc = roc_auc_score(y_val, preds_prob)
            auroc_scores.append(fold_auroc)
            losses.append(log_loss(y_val, preds_prob))

            fpr, tpr, _ = roc_curve(y_val, preds_prob)
```

```
            interp_tpr = np.interp(mean_fpr, fpr, tpr)

            # Anchor array lines via correct vector initialization boundary
            interp_tpr[0] = 0.0
            tprs.append(interp_tpr)

            plt.plot(fpr, tpr, lw=1, alpha=0.5, label=f'Fold {fold+1} (AUC = {fold_auro

        mean_tpr = np.mean(tprs, axis=0)
        mean_tpr[-1] = 1.0
        mean_auc = auc(mean_fpr, mean_tpr)

        plt.plot(mean_fpr, mean_tpr, color='blue', label=f'Mean ROC (AUC = {mean_auc:.2
        plt.plot([0, 1], [0, 1], linestyle='--', color='red', label='Chance Bound')
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title('Microbiome Pipeline Cross-Validation Summary')
        plt.legend(loc="lower right")
        plt.tight_layout()
        plt.show()

        # Compute and render tree model explanations
        explainer = shap.TreeExplainer(self.model)
        shap_values = explainer(X)

        plt.figure(figsize=(10, 6))
        shap.summary_plot(shap_values, X, show=False)
        plt.title("Biological Taxa Impact Analysis (SHAP Value Summary)", fontsize=12,
        plt.tight_layout()
        plt.show()

        return {"mean_auroc": float(np.mean(auroc_scores)), "mean_logloss": float(np.me

if __name__ == "__main__":
    # Execute the generic remote data handler structure
    pipeline = GitHubMicrobiomePipeline(random_state=42)
    X, y = pipeline.fetch_and_pivot_github_data()
    metrics = pipeline.run_cv_and_plots(X, y)
    print(f"\nExecution Complete | Real GitHub Validation AUROC Score: {metrics['mean_a
```

```
--------------------------------------------------------------------------
gaierror                                  Traceback (most recent call last)
/usr/lib/python3.13/urllib/request.py in do_open(self, http_class, req,
**http_conn_args)
   1330             try:
-> 1331                 h.request(req.get_method(), req.selector, req.data, headers,
   1332                           encode_chunked=req.has_header('Transfer-encoding'))

                              23 frames
gaierror: [Errno -5] No address associated with hostname

During handling of the above exception, another exception occurred:

URLError                                  Traceback (most recent call last)
/usr/lib/python3.13/urllib/request.py in do_open(self, http_class, req,
**http_conn_args)
   1332                           encode_chunked=req.has_header('Transfer-encoding'))
   1333             except OSError as err: # timeout error
-> 1334                 raise URLError(err)
   1335             r = h.getresponse()
   1336         except:

URLError: <urlopen error [Errno -5] No address associated with hostname>
```

```
import logging
from typing import Tuple, Dict
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, log_loss, roc_curve, auc
import shap
import matplotlib.pyplot as plt

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s
logger = logging.getLogger(__name__)

class AmericanGutProjectPipeline:
    """
    Pipeline optimized for parsing real-world operational matrices from the
    American Gut Project (PRJEB11419) cohort, executing CV and SHAP diagnostics.
    """

    def __init__(self, random_state: int = 42) -> None:
        self.random_state: int = random_state
        self.model = GradientBoostingClassifier(
            n_estimators=120,
            learning_rate=0.04,
            max_depth=4,
            subsample=0.85,
            random_state=self.random_state
        )
        self.feature_names: list[str] = []

    def load_american_gut_matrix(self, file_path: str, target_phenotype: str) -> Tuple[pd
        """Loads wide-format microbiome taxonomy abundance matrix mapped to clinical des
        logger.info(f"Ingesting American Gut operational cohort data from: {file_path}")
        df = pd.read_csv(file_path)

        if target_phenotype not in df.columns:
            raise KeyError(f"Target clinical phenotype classification vector '{target_ph
```

```
        y = df[target_phenotype].values
        X = df.drop(columns=[target_phenotype])

        # Safe structural type conversion for dynamic relative species counts
        X = X.apply(pd.to_numeric, errors='coerce').fillna(0.0)
        self.feature_names = list(X.columns)

        logger.info(f"Ingestion matching complete. Matrix dimensions: {X.shape} patient |
        return X, y

    def evaluate_cohort_performance(self, X: pd.DataFrame, y: np.ndarray, n_splits: int =
        """Runs rigorous Stratified Cross-Validation and generates publication-grade ROC
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=self.random_s
        auroc_scores = []
        losses = []

        plt.figure(figsize=(8, 6))
        mean_fpr = np.linspace(0, 1, 100)
        tprs = []

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            self.model.fit(X_train, y_train)
            preds_prob = self.model.predict_proba(X_val)[:, 1]

            fold_auroc = roc_auc_score(y_val, preds_prob)
            auroc_scores.append(fold_auroc)
            losses.append(log_loss(y_val, preds_prob))

            fpr, tpr, _ = roc_curve(y_val, preds_prob)
            interp_tpr = np.interp(mean_fpr, fpr, tpr)
            interp_tpr[0] = 0.0  # Completely resolved scalar error via direct array tra
            tprs.append(interp_tpr)

            plt.plot(fpr, tpr, lw=1, alpha=0.5, label=f'Fold {fold+1} (AUC = {fold_auroc

        mean_tpr = np.mean(tprs, axis=0)
        mean_tpr[-1] = 1.0
        mean_auc = auc(mean_fpr, mean_tpr)

        plt.plot(mean_fpr, mean_tpr, color='blue', label=f'Mean ROC (AUC = {mean_auc:.2f}
        plt.plot([0, 1], [0, 1], linestyle='--', color='red', label='Chance Bound')
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title('PRJEB11419 Cohort Performance Validation Summary')
        plt.legend(loc="lower right")
        plt.tight_layout()
        plt.show()

        # Build explainable AI attributions for the real bacteria columns
        explainer = shap.TreeExplainer(self.model)
        shap_values = explainer(X)

        plt.figure(figsize=(10, 6))
        shap.summary_plot(shap_values, X, show=False)
        plt.title("American Gut Taxonomic Drivers Analysis (SHAP Summary)", fontsize=12,
        plt.tight_layout()
        plt.show()
```

```
        return {"mean_auroc": float(np.mean(auroc_scores)), "mean_logloss": float(np.mea

if __name__ == "__main__":
    # Setup native ecosystem mock matrix to simulate the saved local American Gut file f
    mock_species = [
        'Faecalibacterium_prausnitzii', 'Bacteroides_thetaiotaomicron',
        'Bifidobacterium_adolescentis', 'Akkermansia_muciniphila', 'Neurological_Conditi
    ]
    np.random.seed(42)
    mock_matrix = np.random.uniform(0.01, 20.0, size=(120, 5))
    mock_matrix[:, 4] = (mock_matrix[:, 0] * -1.8 + mock_matrix[:, 1] * 1.5 > 1.2).astyp

    pd.DataFrame(mock_matrix, columns=mock_species).to_csv("american_gut_processed_count

    # Execution block targeting the local filesystem matrices
    pipeline = AmericanGutProjectPipeline(random_state=42)
    X, y = pipeline.load_american_gut_matrix(
        file_path="american_gut_processed_counts.csv",
        target_phenotype="Neurological_Condition_Label"
    )
```